

Direttive al preprocessore

Prof. Rizzotti

1 Le direttive al preprocessore

Quando scriviamo un programma in C++, possiamo utilizzare particolari istruzioni che iniziano con il simbolo #.

Ad esempio:

```
1 #include <iostream>
2 #define PI_GRECO 3.14
```

Queste istruzioni prendono il nome di **direttive al preprocessore**.

Il **preprocessore** è uno strumento che interviene prima della compilazione vera e propria del programma.

Il suo compito è leggere il nostro codice sorgente e risolvere le direttive che incontra.

In altre parole, il preprocessore prepara il codice sorgente prima che venga elaborato dal compilatore.

In questa lezione analizzeremo due direttive:

- `#include`
- `#define`

2 La direttiva `#include`

La direttiva `#include` permette di includere nel nostro programma del codice sorgente scritto da altri programmatori.

Questo codice è spesso organizzato in librerie, che mettono a disposizione funzionalità utili per realizzare i nostri programmi.

Ad esempio, abbiamo già utilizzato:

```
1 #include <iostream>
```

Il file di intestazione `iostream` fa parte della libreria standard del C++ e mette a disposizione strumenti per effettuare operazioni di input e output.

Tra questi troviamo:

- `std::cout`, per visualizzare informazioni sullo schermo;
- `std::cin`, per leggere informazioni inserite dall'utente.

2.1 Cosa fa il preprocessore?

Quando il preprocessore incontra una direttiva `#include`, inserisce il contenuto del file indicato all'interno del nostro codice sorgente.

Possiamo immaginare che il preprocessore **incolli il contenuto di altri file all'interno del nostro programma**.

Consideriamo il seguente esempio:

```
1 #include <iostream>
2
3 int main() {
4
5     std::cout << "Hello world!";
6
7     return 0;
8 }
```

Quando il preprocessore incontra:

```
1 #include <iostream>
```

recupera il contenuto del file di intestazione `iostream` e lo inserisce nel nostro codice sorgente.

Il risultato è, concettualmente, un grande file che contiene sia il nostro codice sia il codice proveniente dai file inclusi.

Prima del preprocessore

```
1 #include <iostream>
2
3 int main() {
4
5     std::cout << "Hello world!";
6
7     return 0;
8 }
```

Dopo il preprocessore

```
1 // Contenuto del file iostream
2 // e degli altri file da esso inclusi
3
4 // ...
5
6 // Il nostro codice
7
8 int main() {
9
```

```

10     std::cout << "Hello world!";
11
12     return 0;
13 }

```

Naturalmente, il codice reale incluso attraverso `iostream` è molto più lungo e complesso di quello rappresentato nell'esempio.

Il risultato del preprocessore può essere salvato in un file con estensione `.i`.

Osservando questo file, possiamo trovare il nostro codice insieme al contenuto dei file inclusi.

Attenzione: la direttiva `#include` inserisce il contenuto dei file di intestazione, ma non necessariamente tutto il codice che implementa le funzioni della libreria.

3 La direttiva `#define`

Un'altra importante direttiva al preprocessore è `#define`.

Questa direttiva permette di associare un nome a una sequenza di caratteri.

Quando il preprocessore incontra quel nome nel codice, lo sostituisce con il contenuto specificato nella direttiva.

3.1 Sintassi

La sintassi generale è:

```

1 #define NOME valore

```

Ad esempio:

```

1 #define PI_GRECO 3.14

```

In questo caso, ogni volta che il preprocessore incontra `PI_GRECO`, lo sostituisce con `3.14`.

3.2 Esempio completo

Consideriamo il seguente programma:

```

1 #include <iostream>
2
3 #define PI_GRECO 3.14
4
5 int main() {
6
7     double raggio = 5;
8
9     double area = PI_GRECO * raggio * raggio;
10
11     std::cout << area;

```

```
12  
13     return 0;  
14 }
```

Durante la fase di preprocessing, la direttiva:

```
1 #define PI_GRECO 3.14
```

fa sì che l'istruzione:

```
1 double area = PI_GRECO * raggio * raggio;
```

venga trasformata in:

```
1 double area = 3.14 * raggio * raggio;
```

Il compilatore riceverà quindi il codice dopo questa sostituzione.

1

¹Parti di questo documento sono output di LLM.